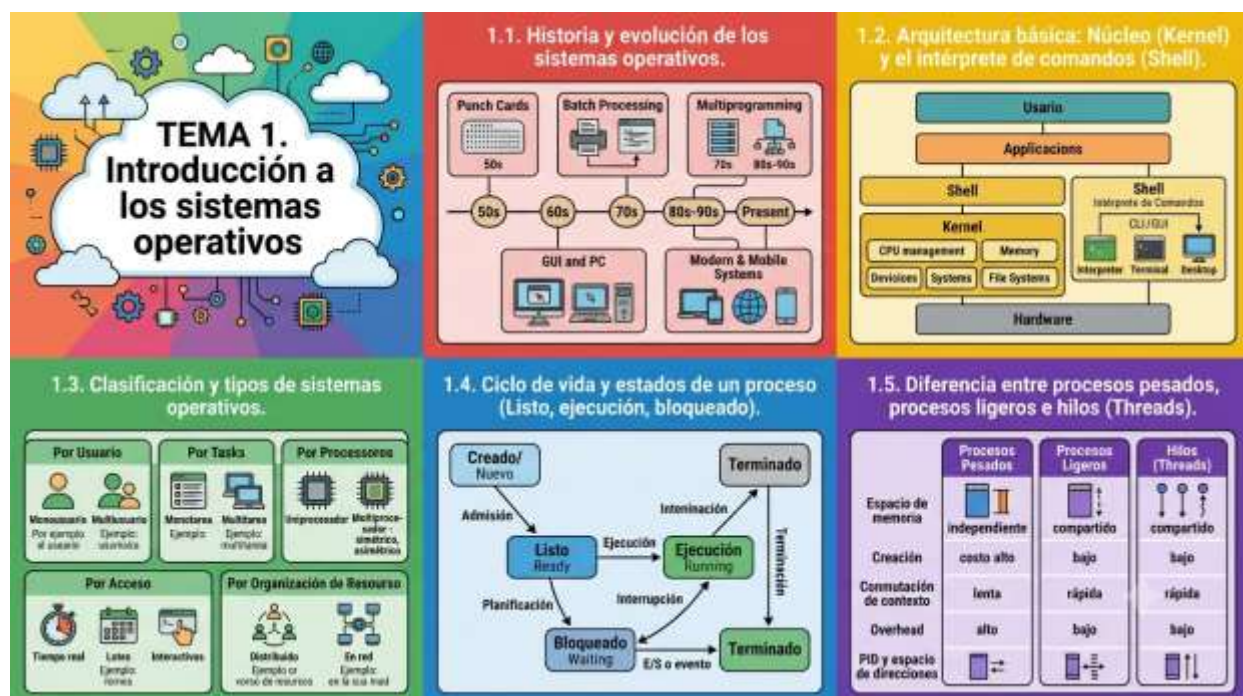


TEMA N° 1



TEMA 1 INTRODUCCIÓN A LOS SISTEMAS OPERATIVOS



EN ESTE TEMA SERÁS CAPAZ DE...

1. **Comprender la evolución histórica y el propósito fundamental** de un sistema operativo, reconociendo cómo transformó el hardware bruto en una herramienta accesible para resolver problemas cotidianos y comerciales.
2. **Identificar y diferenciar los componentes de la arquitectura básica**, específicamente el núcleo (*Kernel*) y el intérprete de comandos (*Shell*), entendiendo cómo interactúan para coordinar los recursos de una computadora.
3. **Analizar y controlar el ciclo de vida de los procesos y sus hilos**, utilizando comandos de consola para observar cómo el sistema operativo administra el tiempo del procesador, optimizando el rendimiento de los equipos informáticos en entornos laborales reales.



PRÁCTICA



PREGUNTA DETONANTE

¿Qué pasaría si al encender una de las computadoras del nuevo módulo educativo del CEA, la pantalla se quedara en negro mostrando únicamente componentes electrónicos internos expuestos, obligándote a escribir códigos en sistema binario (ceros y unos) solo para abrir un documento de texto o calcular las ventas de una tienda?

PLANTEAMIENTO DEL PROBLEMA TECNOLÓGICO REAL

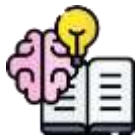
Imagina que te encuentras en el flamante y bien equipado módulo educativo del **CEA Herman Ortiz Camargo**, ubicado en el **Barrio Saó** (a unas 5 cuadras del centro de la población de Pailón). Como futuro **Técnico en Sistemas Informáticos**, la dirección del plantel te encomienda apoyar a la comunidad local mediante soporte técnico básico.

Te presentan tres situaciones simultáneas el mismo día:

1. **Don Amado**, un comerciante del **Barrio Central Fátima**, llega muy preocupado porque la computadora donde registra las cuentas de su tienda de abarrotes (cerca de la iglesia) se ha vuelto extremadamente lenta. Menciona que cuando abre el programa de inventarios, la lectora de códigos de barra deja de responder y el sistema se congela por completo.
2. **La Asociación de Transportistas**, cuya oficina operativa está en el **Barrio 24 de Septiembre** (cerca de la estación de tren y los campos deportivos), informa que su terminal informática no arranca. Al encenderla, muestra un mensaje que dice "*Operating System Not Found*" (Sistema operativo no encontrado), impidiendo que registren la salida de los camiones de soya.
3. **El Profesor de Educación Física**, que reside en el **Barrio 27 de Mayo** (cerca del coliseo municipal), necesita instalar un sistema automatizado para el fixture de un campeonato interbarrial, pero la computadora del coliseo solo tiene 2 Gigabytes de memoria RAM y el programa que compró exige el doble de recursos informáticos para funcionar.



Como estudiante del CEA que está iniciando su formación técnica, observas que el hardware está físicamente intacto en todos los casos: las pantallas encienden, los discos duros giran y los cables están conectados. Sin embargo, los usuarios no pueden interactuar con las máquinas. No hay un puente de comunicación entre las necesidades de estas personas de Pailón y las señales eléctricas de los circuitos de las computadoras. ¿Qué es exactamente lo que está fallando o careciendo en estos dispositivos? ¿Cómo coordina una computadora sus recursos internos para que un comerciante, un transportista o un profesor puedan realizar sus tareas diarias sin saber electrónica digital?



TEORÍA



1.1. HISTORIA Y EVOLUCIÓN DE LOS SISTEMAS OPERATIVOS

Para entender el estado actual de las computadoras del **CEA Herman Ortiz Camargo**, es indispensable viajar al pasado y comprender cómo la informática dejó de ser una disciplina de laboratorios científicos para convertirse en una herramienta de uso diario en Pailón.

En los inicios de la informática moderna (década de 1940 y 1950), las computadoras no poseían un sistema operativo. Los programadores debían interactuar directamente con el hardware utilizando cables y conmutadores físicos. Una sola persona ocupaba toda una habitación para ejecutar un único programa de cálculo matemático. Esto se conoce como la **Generación Cero**. El desperdicio de tiempo del procesador era inmenso, ya que la máquina permanecía inactiva mientras el operador montaba las cintas magnéticas o acomodaba pilas de tarjetas perforadas.

Con la llegada de la primera generación de sistemas operativos en la década de 1960, se implementó el **Procesamiento por Lotes (Batch Processing)**. Los Técnicos reunían varios trabajos similares en una cinta y la computadora los ejecutaba uno tras otro de forma secuencial, sin interrupción. Si un programa fallaba en medio del lote, la máquina se detenía por completo, afectando a los demás usuarios.



La verdadera revolución llegó con la **Multiprogramación** y el **Tiempo Compartido** en los años 1970 y 1980. Los sistemas operativos como UNIX permitieron que la memoria central albergara varios programas simultáneamente. El procesador alternaba de manera increíblemente rápida entre un usuario y otro, creando la ilusión de que cada persona disponía de la computadora en exclusiva. Esta evolución dio origen a los sistemas operativos de escritorio que conocemos hoy, como Microsoft Windows, macOS y las diversas distribuciones libres de Linux, los cuales gestionan de manera transparente los recursos físicos del equipo informático.

1.2. ARQUITECTURA BÁSICA: NÚCLEO (KERNEL) Y EL INTÉRPRETE DE COMANDOS (SHELL)

Un sistema operativo no es un bloque de software único y monolítico; se organiza en capas concéntricas de abstracción. El diseño clásico divide el sistema en dos componentes fundamentales que operan en perfecta sintonía: el **Núcleo (Kernel)** y el **Intérprete de Comandos (Shell)**.

El **Núcleo o Kernel** es la parte más interna y crítica del sistema operativo. Se ejecuta en el denominado "modo supervisor" o "modo núcleo", teniendo acceso directo e irrestricto a todo el hardware de la computadora (Procesador, Memoria RAM, Discos Duros, Tarjetas de Red). Sus funciones principales incluyen:



1. **Gestión de Memoria:** Asignar y proteger los bloques de memoria RAM para que el programa de un usuario no invada ni corrompa el espacio de memoria de otro programa.
2. **Administración del Procesador:** Decidir qué proceso tiene derecho a usar el CPU y durante cuántos milisegundos (Planificación de Procesos).
3. **Controladores de Dispositivos (Drivers):** Traducir las instrucciones genéricas del software en comandos eléctricos específicos que los periféricos (como la lectora de códigos de barra del Barrio Central Fátima) puedan comprender.

Por otro lado, el **Intérprete de Comandos o Shell** es la capa externa que actúa como interfaz entre el usuario y el Kernel. El Kernel es completamente ciego a las intenciones humanas directas; requiere que el Shell reciba las órdenes del usuario, las valide sintácticamente y solicite los servicios correspondientes al Núcleo a través de **Llamadas al Sistema (System Calls)**.

El Shell puede presentarse bajo dos modalidades principales:

1. **CLI (Command Line Interface - Interfaz de Línea de Comandos):** Una pantalla de texto donde el Técnico escribe comandos específicos (ej. Bash en Linux o PowerShell en Windows).
2. **GUI (Graphical User Interface - Interfaz Gráfica de Usuario):** Un entorno visual con ventanas, íconos y punteros de mouse, diseñado para facilitar el acceso a usuarios no técnicos.

A continuación, se presenta un ejemplo de un script básico escrito para el Shell de Linux (Bash) que un Técnico puede ejecutar en el laboratorio del CEA para auditar la información del hardware que gestiona el Kernel:

Bash

```
#!/bin/bash
# Este script automatiza la consulta de recursos básicos del sistema operativo
echo "=== Reporte de Sistema para el CEA Herman Ortiz Camargo ==="
# Muestra el nombre del sistema operativo y la versión del Kernel en uso
uname -a
echo "=== Uso de Memoria RAM en tiempo real ==="
```



```
# EL comando '
free -m
' solicita al Kernel el estado de la memoria RAM en Megabytes
free -m

echo "=== Espacio disponible en las unidades de almacenamiento ==="
# '
df -h
' interroga al Kernel sobre los discos montados en formato legible para humanos
df -h

echo "=====
```



1.3. CLASIFICACIÓN Y TIPOS DE SISTEMAS OPERATIVOS

Como Técnico en Sistemas Informáticos, te encontrarás con una gran variedad de escenarios comerciales y educativos en Pailón. Para seleccionar el software adecuado, debes conocer la clasificación técnica de los sistemas operativos según sus capacidades de administración:

a) Según la administración de usuarios:

1. **Monousuario:** Diseñados para atender a un solo usuario a la vez. Los sistemas operativos de teléfonos móviles convencionales o las computadoras de escritorio domésticas entran en esta categoría, donde un solo individuo mantiene el control de la sesión de trabajo.



2. **Multiusuario:** Capaces de autenticar y procesar las peticiones de decenas, cientos o miles de usuarios simultáneamente, protegiendo la privacidad y los archivos de cada uno. Son utilizados en los servidores de bases de datos o plataformas web académicas.

b) Según la administración de tareas:

1. **Monotarea:** El sistema operativo solo puede ejecutar un proceso a la vez. Si estás imprimiendo un reporte de notas, no puedes redactar un texto de forma simultánea; debes esperar a que la primera tarea finalice por completo (como sucedía en el antiguo MS-DOS).
2. **Multitarea:** Permite que el procesador ejecute múltiples programas concurrentemente. El sistema operativo distribuye el tiempo del CPU en fracciones tan pequeñas que para el Técnico que trabaja en el Barrio Residencial Norte le parece que su reproductor de música, su navegador web y su entorno de desarrollo se ejecutan exactamente al mismo tiempo.

c) Según la estructura de su diseño interno:

1. **Monolíticos:** Todo el sistema operativo (planificación, archivos, memoria, drivers) se ejecuta dentro de un único espacio de memoria protegido de gran tamaño. Es extremadamente rápido pero un fallo en un driver puede congelar todo el sistema.
2. **Microkernel:** El núcleo se reduce a sus funciones mínimas indispensables (comunicación e hilos básicos). Los demás servicios (como el sistema de archivos o los controladores de red) se ejecutan como programas de usuario independientes. Si un driver falla, se reinicia de forma aislada sin tumbar todo el sistema operativo.

1.4. CICLO DE VIDA Y ESTADOS DE UN PROCESO (LISTO, EJECUCIÓN, BLOQUEADO)

Un concepto central en el Taller de Sistemas Operativos es comprender qué es un **Proceso**. Un proceso no es simplemente un programa guardado en el disco duro; un proceso es un **programa en ejecución**, lo que significa que posee memoria asignada, registros en el CPU y acceso a recursos del sistema.

Para que el procesador de un solo núcleo pueda atender múltiples demandas en los negocios del Barrio Central Fátima o las oficinas de las terminales de Pailón, el sistema operativo



implementa un modelo de estados dinámico controlado por el **Planificador de Procesos (Scheduler)**. Un proceso pasa de forma obligatoria por los siguientes estados fundamentales durante su existencia:

1. **Nuevo (New):** El programa está siendo cargado desde el almacenamiento secundario (Disco Duro o SSD) hacia la memoria RAM. El sistema operativo inicializa las estructuras de control correspondientes.
2. **Listo (Ready):** El proceso está completamente cargado en la memoria RAM y posee todos los recursos necesarios para trabajar, pero está esperando pacientemente a que el Planificador de Procesos le asigne un turno en el CPU.
3. **Ejecución (Running):** El procesador ha tomado las instrucciones del proceso y las está procesando activamente a la velocidad de sus ciclos de reloj.
4. **Bloqueado / Espera (Blocked / Waiting):** El proceso se ve obligado a pausar su ejecución en el CPU porque requiere un evento externo para continuar. Por ejemplo, está esperando a que el operador presione una tecla, que llegue un dato desde la red de internet de Pailón, o que el disco duro termine de leer un archivo pesado. Mientras está bloqueado, libera el CPU para que otros procesos avancen.
5. **Terminado (Terminated):** El proceso finalizó sus instrucciones o fue cerrado por el usuario. El sistema operativo libera toda la memoria RAM y los recursos que tenía asignados para que queden disponibles para el resto del sistema.





Las transiciones entre estados ocurren continuamente miles de veces por segundo:

1. De *Listo* a *Ejecución* ocurre mediante el **Despacho (Dispatch)** del Scheduler.
2. De *Ejecución* a *Listo* ocurre por **Fin de Tiempo (Timeout)**, cuando el sistema operativo le retira el uso del CPU porque se agotó su cuota de milisegundos asignada (Quantum), permitiendo que otra aplicación use la máquina.
3. De *Ejecución* a *Bloqueado* ocurre por una solicitud de **Entrada/Salida (I/O Wait)**.
4. De *Bloqueado* a *Listo* ocurre cuando el evento externo concluye satisfactoriamente (**I/O Completion**), regresando a la fila de espera para volver a usar el procesador.

1.5. DIFERENCIA ENTRE PROCESOS PESADOS, PROCESOS LIGEROS E HILOS (THREADS)

En el desarrollo de software y en la administración de servidores modernos, los Técnicos deben optimizar al máximo el uso de la memoria RAM y el procesador. Para ello, es vital entender cómo se subdividen las unidades de ejecución de un programa informático.

Un **Proceso Pesado (Proceso Tradicional)** representa una instancia de ejecución totalmente aislada e independiente. Tiene su propio espacio de memoria virtual protegido, sus propias variables y sus propios descriptores de archivos. Crear un proceso pesado consume bastantes recursos del sistema operativo porque el Kernel debe reservar bloques enteros de memoria RAM exclusivos. Si un proceso pesado falla o se cuelga (por ejemplo, debido a un error de programación), los demás procesos pesados del sistema continúan funcionando con total normalidad porque están aislados por hardware.

Un **Hilo o Proceso Ligero (Thread)** es la unidad más pequeña de ejecución que el sistema operativo puede planificar de forma independiente. Un proceso pesado puede contener múltiples hilos en su interior. La característica fundamental de los hilos es que **comparten el mismo espacio de memoria RAM y los mismos recursos** asignados al proceso padre que los originó.

A continuación se detallan las diferencias técnicas críticas:



Característica	Proceso Pesado	Hilo / Proceso Ligero (Thread)
Espacio de Memoria	Totalmente aislado e independiente por cada proceso.	Compartido entre todos los hilos del mismo proceso.
Consumo de Recursos	Elevado en CPU y Memoria RAM al crearse o destruirse.	Muy bajo y ligero; ideal para operaciones concurrentes rápidas.
Comunicación	Requiere mecanismos complejos (IPC, Sockets, Tuberías).	Directa y veloz a través de las variables compartidas en memoria.
Impacto ante Errores	Si falla, solo se cierra ese proceso en particular.	Si un hilo sufre un error crítico, puede tumbar a todo el proceso padre.

Para visualizarlo prácticamente: Imagina una aplicación de mensajería instalada en una computadora del Barrio María Belén. El programa completo es un **Proceso Pesado**. Dentro de él, se crean tres **Hilos**: un hilo se encarga exclusivamente de renderizar la interfaz gráfica en la pantalla para que el usuario mueva el mouse sin retrasos, otro hilo se dedica a escuchar la llegada de nuevos mensajes desde el servidor de internet, y un tercer hilo escribe el historial de conversaciones de forma asíncrona en el almacenamiento interno del equipo informático.

1.6. SISTEMAS OPERATIVOS MODERNOS EN LA NUBE Y VIRTUALIZACIÓN

En la actualidad, la labor de un Técnico va mucho más allá de dar mantenimiento a computadoras físicas locales en Pailón. Con el auge de la computación en la nube, los sistemas operativos se han adaptado para operar en entornos altamente abstractos y distribuidos.

La **Virtualización** es la tecnología que permite ejecutar múltiples sistemas operativos independientes sobre un único hardware físico real. Esto se logra mediante una capa de software especializada denominada **Hipervisor o Monitor de Máquina Virtual (VMM)**. El hipervisor engaña a los sistemas operativos invitados haciéndoles creer que disponen de procesadores y memorias RAM físicas dedicadas, cuando en realidad están compartiendo los recursos de un único servidor físico robusto.

Existen dos tipos principales de hipervisores:



1. **Tipo 1 (Nativo o Bare-Metal):** Se instala directamente sobre el hardware físico de la máquina, sin necesidad de un sistema operativo previo. Son extremadamente eficientes y se emplean en grandes centros de datos corporativos.
2. **Tipo 2 (Hosted o Alojado):** Se ejecuta como una aplicación ordinaria encima de un sistema operativo anfitrión convencional (por ejemplo, instalar Oracle VirtualBox sobre Windows 11 en las computadoras del laboratorio del CEA). Es ideal para fines educativos y pruebas técnicas rápidas de compatibilidad de software.

Adicionalmente, la tendencia contemporánea apunta hacia la **Contenedorización** (liderada por herramientas como Docker). A diferencia de una máquina virtual clásica que emula un sistema operativo completo con su propio Kernel pesado, un contenedor comparte el mismo Kernel del sistema operativo anfitrión, aislando únicamente las librerías y dependencias específicas del programa del usuario. Esto permite desplegar aplicaciones web en segundos consumiendo una fracción mínima de memoria RAM.

1.7. SISTEMAS OPERATIVOS EMBEBIDOS Y EL INTERNET DE LAS COSAS (IOT)

Cuando miramos a nuestro alrededor en los diferentes distritos de Pailón, notamos que las computadoras ya no solo tienen forma de torres de escritorio o laptops de oficina. Hoy en día, encontramos chips inteligentes controlando los semáforos, los sistemas de riego agrícola automatizados en los campos aledaños y los equipos médicos de los centros de salud locales. Estos dispositivos funcionan gracias a los **Sistemas Operativos Embebidos o Integrados**.

Un sistema operativo embebido está diseñado específicamente para realizar una tarea de control fija y dedicada, optimizando al máximo recursos de hardware sumamente limitados (procesadores de pocos Megahertz y memorias medidas en Kilobytes). A menudo, estos dispositivos requieren operar bajo la modalidad de **Sistemas Operativos en Tiempo Real (RTOS - Real-Time Operating Systems)**.

En un sistema operativo convencional (como Windows o Linux de escritorio), si abres muchas pestañas en el navegador, el sistema puede ralentizarse unos segundos sin que ocurra una catástrofe. En un RTOS, el factor crítico es la **previsibilidad del tiempo**: las operaciones deben ejecutarse estrictamente dentro de un límite de tiempo exacto y garantizado. Si el sistema



operativo embebido que controla los frenos ABS de un vehículo o las válvulas de presión de un silo de granos en Pailón se retrasa un milisegundo en procesar una interrupción del hardware, el resultado puede ser un fallo crítico o un accidente grave. Ejemplos comunes de estos entornos incluyen FreeRTOS, Embedded Linux y las arquitecturas simplificadas para microcontroladores industriales.

1.8. LA INTEGRACIÓN DE LA INTELIGENCIA ARTIFICIAL EN LA GESTIÓN DEL SISTEMA OPERATIVO

La última frontera tecnológica en el desarrollo de sistemas operativos es la incorporación de núcleos inteligentes capaces de auto-optimizarse en tiempo real sin la intervención constante de un Administrador de Sistemas o Técnico humano.

Tradicionalmente, los algoritmos de planificación de procesos y gestión de memoria RAM del Kernel se basaban en reglas estáticas preprogramadas (como políticas FIFO: Primero en entrar, primero en salir, o algoritmos de prioridad fija). Los sistemas operativos modernos de última generación están integrando modelos predictivos ligeros basados en **Inteligencia Artificial y Aprendizaje Automático (Machine Learning)** directamente en las capas del Kernel.

Esta integración permite las siguientes mejoras de rendimiento revolucionarias:

1. **Planificación Predictiva del CPU:** El Kernel analiza los patrones históricos de comportamiento del usuario en Pailón. Si detecta que a las 19:00 el estudiante del CEA siempre abre el entorno de desarrollo de código, el sistema operativo preasigna recursos de procesamiento de forma anticipada.
2. **Gestión Inteligente de Memoria RAM:** En lugar de esperar a que la memoria RAM se llene por completo para empezar a paginar datos hacia el disco de almacenamiento, la IA predice qué bloques de memoria no serán utilizados en el futuro cercano y los libera de manera proactiva.
3. **Seguridad y Detección de Anomalías:** El sistema operativo monitorea constantemente las llamadas al sistema (*System Calls*). Si un hilo de ejecución comienza a solicitar accesos a archivos de forma inusual a una velocidad sospechosa, la IA del Kernel identifica el patrón



como un posible ataque de *Ransomware* o virus informático, bloqueando el proceso de forma inmediata antes de que logre encriptar la información del usuario.

CIERRE TEÓRICO OBLIGATORIO

Mitos vs. Realidades

Mito	Realidad
"Cuanta más memoria RAM libre muestre el administrador de tareas del sistema operativo, significa que la computadora funcionará mucho más rápido en las oficinas de Pailón."	La memoria RAM que permanece vacía es memoria desperdiciada. Los sistemas operativos modernos usan la RAM libre para almacenar en caché los archivos más utilizados, acelerando el sistema. El problema real ocurre cuando la RAM se satura al 100% y el sistema debe usar el almacenamiento secundario lento.
"Al cerrar la ventana de una aplicación haciendo clic en la 'X', todos sus componentes e hilos internos se borran instantáneamente de la memoria del computador de manera limpia."	Muchas aplicaciones modernas cierran su interfaz gráfica (Shell), pero dejan procesos secundarios e hilos ejecutándose en segundo plano (Modo Bloqueado o Listo oculto) para buscar actualizaciones o enviar notificaciones, consumiendo recursos valiosos del procesador de forma silenciosa.

Glosario de Términos

1. **Kernel (Núcleo):** El componente de software central e indispensable de un sistema operativo que gestiona directamente las comunicaciones entre el hardware físico y los programas informáticos del usuario.
2. **Shell (Interfaz):** La capa de software externa del sistema operativo que actúa como puente de comunicación, permitiendo a los usuarios enviar órdenes legibles al Kernel mediante texto (CLI) o gráficos (GUI).
3. **Proceso:** Una instancia de un programa informático que se encuentra cargada activamente en la memoria RAM y está siendo ejecutada por el procesador, disponiendo de recursos y un estado propio.



4. **Hilo (Thread):** La unidad básica y más liviana de ejecución contenida dentro de un proceso pesado, capaz de compartir la misma memoria RAM con otros hilos del mismo proceso padre para agilizar el trabajo.
5. **Llamada al Sistema (System Call):** El mecanismo de software estructurado que utilizan las aplicaciones de usuario para solicitar formalmente servicios de control o acceso al hardware directamente al Kernel del sistema operativo.

▣ **Ponte a Prueba**

1. **Cuando un proceso se encuentra cargado en la memoria RAM con todos sus recursos listos, pero aguarda a que el planificador le asigne un turno en el procesador, ¿en qué estado específico del ciclo de vida se encuentra localizado?**

1. A) Ejecución
2. B) Bloqueado
3. C) Listo
4. D) Nuevo

2. **¿Cuál es la principal ventaja arquitectónica de utilizar Hilos (Threads) en lugar de múltiples Procesos Pesados para realizar tareas concurrentes en un programa de software?**

1. A) Brindan un aislamiento total por hardware que impide que cualquier error afecte a otros hilos.
2. B) Comparten el mismo espacio de memoria RAM, lo que reduce el consumo de recursos informáticos y acelera la comunicación interna.
3. C) Permiten prescindir por completo de los servicios de control del Kernel del sistema operativo.
4. D) Son de tipo monolítico y se ejecutan exclusivamente en la unidad de almacenamiento secundario.



3. Un Técnico que trabaja en las computadoras del Barrio Central Fátima necesita interactuar de forma directa con el sistema operativo escribiendo comandos de texto puro en una consola negra. ¿Qué capa de la arquitectura básica está operando directamente en ese instante?

1. A) Microkernel
2. B) Hardware Físico
3. C) Shell CLI
4. D) Driver del Periférico

Anexo de respuestas correctoras: 1-C, 2-B, 3-C.



VALORACIÓN



REFLEXIÓN PROFUNDA SOBRE EL IMPACTO ÉTICO, SOCIAL Y MEDIOAMBIENTAL

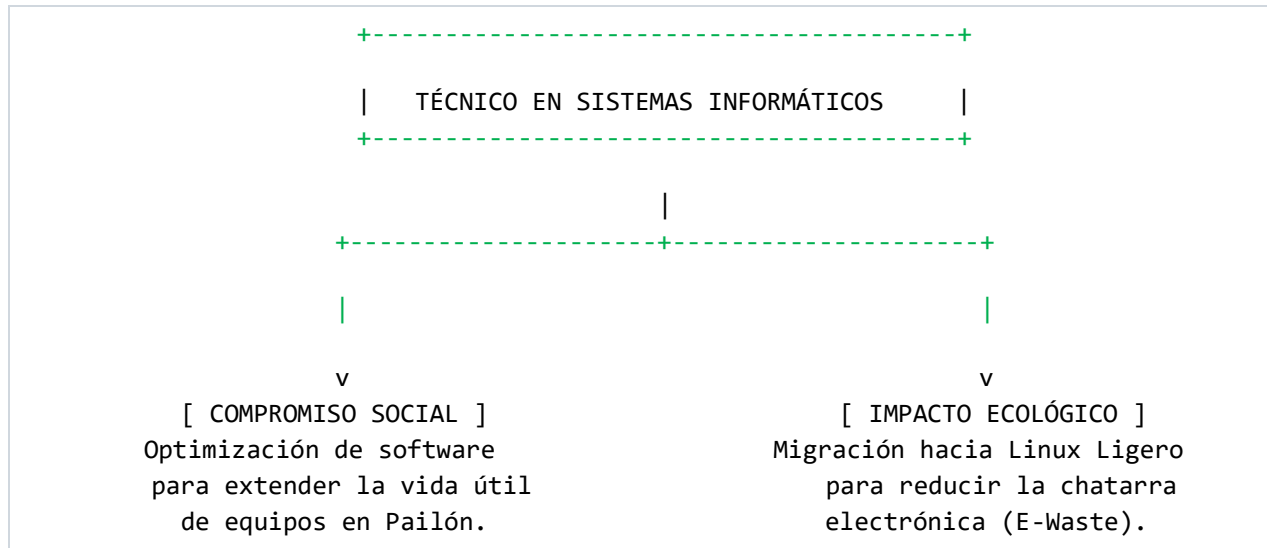
El estudio de los sistemas operativos no debe limitarse al dominio técnico de comandos y configuraciones de hardware en el aula del Barrio Saó. Como Técnicos en Sistemas Informáticos, las decisiones de software que tomen impactan directamente en la sociedad de Pailón y en la sostenibilidad de nuestro medio ambiente global.

En primer lugar, analicemos el problema ético y social del **Acceso Tecnológico y la Obsolescencia Programada**. Las grandes corporaciones de software actualizan sus sistemas operativos comerciales exigiendo continuamente requisitos de hardware más elevados y costosos. Esto provoca la exclusión directa de sectores de la población de Pailón que no cuentan con los recursos financieros para adquirir computadoras de última generación.

Aquí es donde la ética profesional del Técnico cobra un valor fundamental: el conocimiento profundo de la arquitectura de sistemas operativos alternativos y eficientes (como las distribuciones libres de GNU/Linux) les permite recuperar, optimizar y extender la vida útil de computadoras consideradas obsoletas. Al instalar sistemas operativos ligeros en equipos antiguos de escuelas o pequeños comercios del Barrio Central Fátima, democratizan el acceso a



la educación digital y al desarrollo económico regional sin obligar a los usuarios a incurrir en deudas económicas.



En segundo lugar, se debe considerar el alarmante impacto medioambiental de la **Chatarra Electrónica (E-Waste)**. Cada computadora desechada prematuramente porque su sistema operativo propietario ya no recibe soporte técnico se convierte en toneladas de residuos tóxicos que contaminan suelos y acuíferos.

Asimismo, la eficiencia en la gestión de procesos e hilos que realiza un sistema operativo afecta directamente el **consumo energético de los centros de datos y servidores**. Un código ineficiente que mantiene al procesador trabajando innecesariamente al 100% de su capacidad eleva la huella de carbono y el gasto de electricidad. Aprender a monitorear y depurar hilos bloqueados o procesos zombis no es solo una habilidad de rendimiento informático; es un acto de responsabilidad ecológica que todo Técnico comprometido con su comunidad debe aplicar diariamente.



PRODUCCIÓN



LABORATORIO PRÁCTICO: MONITOREO Y CONTROL DE PROCESOS EN ENTORNOS DE ESCRITORIO

Objetivo del Laboratorio

Aplicar de forma práctica los conceptos teóricos aprendidos sobre el ciclo de vida de los procesos, el intérprete de comandos (Shell) y la asignación de recursos mediante la manipulación directa de tareas simuladas en el laboratorio informático del CEA.

Escenario del Reto Técnico

La administración central del nuevo módulo educativo del **CEA Herman Ortiz Camargo** en el **Barrio Saó** reporta que una aplicación web de consultas estudiantiles se ha quedado colgada y está consumiendo memoria en exceso, ralentizando el trabajo administrativo. Tu deber como Técnico es identificar el proceso problemático usando el Shell, analizar su comportamiento y forzar su terminación de manera limpia y segura.

Guía Paso a Paso (Instrucciones para Sistemas Windows / PowerShell)

Paso 1: Apertura del Intérprete de Comandos (Shell)

En la computadora del laboratorio, haz clic en el menú de inicio, escribe PowerShell y presiona Enter. Se abrirá la ventana de texto del Shell CLI listo para recibir tus órdenes.

Paso 2: Listar todos los procesos activos en memoria RAM

Escribe el siguiente comando para interrogar al Kernel sobre todos los programas en ejecución actualmente:

```
PowerShell
```

```
# Get-Process solicita al Kernel la lista completa de procesos cargados en la memoria RAM
```

```
Get-Process
```



Observarás múltiples columnas en la pantalla. Pon especial atención a las columnas tituladas SI (ID del Proceso), CPU(s) (Tiempo de procesador consumido) y WS(K) (Memoria RAM consumida en Kilobytes).

Paso 3: Filtrar la búsqueda para ubicar procesos específicos

Dado que la lista general es muy extensa, filtraremos los resultados para buscar únicamente aplicaciones asociadas al navegador de internet (por ejemplo, Google Chrome o Microsoft Edge) que suelen simular el comportamiento de procesos pesados con múltiples hilos:

PowerShell

```
# Filtra y muestra únicamente los procesos en ejecución cuyo nombre contenga la palabra  
'  
msedge'  
Get-Process -Name msedge
```

Paso 4: Simular la creación de un proceso pesado en segundo plano

Para experimentar sin poner en riesgo los sistemas administrativos del CEA, iniciaremos un proceso ligero e inofensivo (el bloc de notas de Windows) configurado deliberadamente para ejecutarse de fondo de manera controlada:

PowerShell

```
# Start-Process inicia el ejecutable del Bloc de Notas  
# -ArgumentList abre un archivo ficticio simulando datos de notas de Los barrios de Pailón  
# -WindowStyle Hidden simula un proceso en segundo plano que trabaja sin interfaz gráfica visible  
Start-Process notepad.exe -ArgumentList "  
notas_barrio_sao.txt" -WindowStyle Hidden
```

Paso 5: Verificar el nuevo proceso y registrar su identificador único (PID)

Comprueba que el proceso de simulación se encuentra ahora en estado **Listo** o **Ejecución** en la memoria RAM de la máquina:

PowerShell

```
# Busca en la memoria el proceso específico del bloc de notas creado previamente  
Get-Process -Name notepad
```



Anota el número entero que aparece bajo la columna Id. Ese número es el **PID (Process Identifier)**, la cédula de identidad única que el Kernel le asigna a ese proceso para diferenciarlo de todos los demás.

Paso 6: Forzar la terminación del proceso (Simulación de reparación técnica)

Imagina que este proceso se ha bloqueado y congela el sistema del coliseo del Barrio 27 de Mayo. Utiliza el comando de finalización forzada enviando una interrupción directa al Kernel usando el PID que anotaste (reemplaza el número 1234 del ejemplo por tu PID real obtenido en el laboratorio):

PowerShell

```
# Stop-Process envía una señal de terminación crítica al Kernel para el PID especificado
# -Force obliga al sistema a liberar la memoria RAM inmediatamente sin pedir confirmación adicional
Stop-Process -Id 1234 -Force
```

Paso 7: Comprobación final del estado del sistema

Ejecuta nuevamente la búsqueda para certificar que el proceso ha transitado exitosamente al estado **Terminado** y que el Kernel ha limpiado correctamente los registros de la memoria RAM del computador:

PowerShell

```
# Intenta buscar el proceso;
ahora debería lanzar un mensaje de error indicando que el proceso ya no existe
Get-Process -Name notepad
```

Entregable Obligatorio para el Estudiante

Para validar la correcta ejecución de este laboratorio del MOMENTO 4, cada Técnico deberá redactar un informe escrito a mano o digital que incluya:

1. Una captura de pantalla o transcripción del PID asignado por el Kernel a su proceso de prueba en el laboratorio del CEA.
2. Una explicación breve (máximo 5 líneas) con sus propias palabras detallando por qué el comando del Paso 6 es una herramienta de salvamento crítica cuando un negocio en el



Barrio Central Fátima sufre la congelación de sus sistemas informáticos informáticos cotidianos.

RECURSOS ADICIONALES

1. **The Linux Documentation Project (TLDP):** Una de las plataformas de documentación técnica libre más grandes y respetadas del mundo. Es ideal para que los estudiantes profundicen en la arquitectura interna del Kernel Linux, la gestión de descriptores de archivos y la administración avanzada de scripts de automatización para el Shell Bash.
2. **Microsoft Learn - Documentación de Arquitectura de Windows de Escritorio y Servidores:** El repositorio oficial de aprendizaje de Microsoft. Proporciona manuales técnicos detallados y guías paso a paso sobre el funcionamiento interno de la arquitectura de la familia de sistemas operativos Windows NT, explicando de forma didáctica la gestión de hilos, los niveles de prioridad del planificador y el uso exhaustivo de la consola avanzada PowerShell.
3. **GitHub Guides - Repositorio de Scripts y Automatización para Administración de Sistemas Operativos:** Una recopilación de plantillas de código fuente abierto, tutoriales interactivos y ejemplos prácticos de scripts creados por la comunidad global de administradores de sistemas informáticos. Es un recurso excelente para aprender a monitorear hilos de ejecución en tiempo real y optimizar la infraestructura de servidores locales.